

C#.NET Properties



Mr. Bangar Raju

 **NARESH**
technologies

Understanding Properties in C#: A Practical Guide

By Albert Harmon

Published October 26, 2016

🕒 4 min read

Education



TLDR This blog post explores the concept of properties in C#, detailing their implementation, advantages over public variables, and how to enforce access restrictions through properties. It provides a comprehensive example using a customer class to illustrate the practical application of

properties in managing data access and integrity.

Welcome to this comprehensive guide on properties in C#. In this post, we will delve into the practical implementation of properties, building on the foundational concepts introduced in the previous session. We will explore how to define properties, their advantages over public variables, and how to enforce access restrictions effectively.

Recap of Properties

In the first part of our tutorial, we discussed what properties are, why they are essential, and how to define them. Properties in C# are special methods that provide a flexible mechanism to read, write, or compute the values of private fields. They help encapsulate data and control access to it.

Defining a Customer Class

To illustrate the use of properties, we will create a `Customer`` class that represents a bank customer. Each customer will have the following attributes:

- Customer ID
- Customer Name
- Balance

- Status (active or inactive)

Class Definition

```
public class Customer
{
    private int _customerId;
    private string _customerName;
    private double _balance;
    private bool _status;

    public Customer(int customerId, string customerName, double balance, bool status)
    {
        _customerId = customerId;
        _customerName = customerName;
        _balance = balance;
        _status = status;
    }
}
```

In this class, we have defined four private fields to store the customer attributes. The constructor initializes these fields when a new instance of the `Customer` class is created.

Implementing Properties

Next, we will implement properties for each of the attributes. This will allow controlled access to the private fields.

Read-Only Property for Customer ID

The `CustomerID` should be read-only, as it should not be modified after the customer is created.

```
public int CustomerID
{
    get { return _customerId; }
}
```

Read-Write Properties for Other Attributes

For the other attributes, we will define read-write properties:

```
public string CustomerName
{
    get { return _customerName; }
    set { _customerName = value; }
}

public double Balance
{
    get { return _balance; }
    set { _balance = value; }
```

```
}

public bool Status
{
    get { return _status; }
    set { _status = value; }
}
```

Access Control with Properties

One of the significant advantages of using properties is the ability to control access to the data. For instance, we may want to restrict modifications to the `CustomerName` and `Balance` based on the `Status` of the account.

Implementing Conditions in Setters

We can enforce conditions in the setter methods to ensure that modifications are only allowed when the account is active:

```
public string CustomerName
{
    get { return _customerName; }
    set
    {
        if (!_status)
        {
```

```

        _customerName = value;
    }
    else
    {
        throw new InvalidOperationException("Cannot modify
    }
}

public double Balance
{
    get { return _balance; }
    set
    {
        if (_status && value >= 0)
        {
            _balance = value;
        }
        else
        {
            throw new InvalidOperationException("Cannot modify
        }
    }
}

```

Example Usage

Now, let's see how we can use the `Customer` class and its properties in a test class:

```
public class TestCustomer
{
    public static void Main(string[] args)
    {
        Customer customer = new Customer(1, "John Doe", 5000,

        // Attempt to modify the name when the account is inactive
        try
        {
            customer.CustomerName = "John Smith"; // This should s
        }
        catch (InvalidOperationException ex)
        {
            Console.WriteLine(ex.Message);
        }

        // Activate the account and modify the name
        customer.Status = true;
        customer.CustomerName = "John Smith"; // This should s
        Console.WriteLine(customer.CustomerName); // Outputs "

    }
}
```



Conclusion

In this post, we explored the concept of properties in C# and their practical implementation through a `Customer`` class. We learned how to define properties, enforce access restrictions, and control data integrity through conditional logic in property setters. Properties provide a powerful way to encapsulate data and ensure that it is accessed and modified in a controlled manner, making them a crucial aspect of object-oriented programming in C#.

Thank you for following along, and stay tuned for more tutorials on C# and software development!